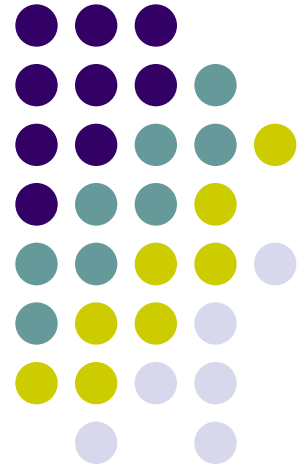


php



Dr:Wafaa Alnbhany

Lecture5



Arithmetic Operators

Operator	Description	Example	Result
+	Addition	x=2 x+2	4
-	Subtraction	x=2 5-x	3
*	Multiplication	x=4 x*5	20
/	Division	15/5 5/2	3 2.5
%	Modulus (division remainder)	5%2 10%8 10%2	1 2 0
++	Increment	x=5 x++	x=6
--	Decrement	x=5 x--	x=4

Comparison Operators



Operator	Description	Example
==	is equal to	5==8 returns false
!=	is not equal	5!=8 returns true
>	is greater than	5>8 returns false
<	is less than	5<8 returns true
>=	is greater than or equal to	5>=8 returns false
<=	is less than or equal to	5<=8 returns true

Logical Operators

Operator	Description	Example
&&	and	x=6 y=3 (x < 10 && y > 1) returns true
	or	x=6 y=3 (x==5 y==5) returns false
!	not	x=6 y=3 !(x==y) returns true

PHP If...Else Statements

The if, elseif and else statements in PHP are used to perform different actions based on different conditions.

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions.

You can use conditional statements in your code to do this.

- **if...else statement** - use this statement if you want to execute a set of code when a condition is true and another if the condition is not true
- **elseif statement** - is used with the if...else statement to execute a set of code if **one** of several condition are true

The If...Else Statement

If you want to execute some code if a condition is true and another code if a condition is false, use the if...else statement.

Syntax

```
if (condition)  
    code to be executed if condition is true;  
else  
    code to be executed if condition is false;
```

Example

The following example will output "Have a nice weekend!" if the current day is Friday, otherwise it will output "Have a nice day!":

```
<html>
<body>
<?php
$d=date("D");
if ($d=="Fri")
    echo "Have a nice weekend!";
else
    echo "Have a nice day!";
?>
</body>
</html>
```

If more than one line should be executed if a condition is true/false, the lines should be enclosed within curly braces:

```
<html>
<body>
<?php
$d=date("D");
if ($d=="Fri")
{
    echo "Hello!<br />";
    echo "Have a nice weekend!";
    echo "See you on Monday!";
}
?>
</body>

</html>
```


The ElseIf Statement

If you want to execute some code if one of several conditions are true use the elseif statement

Syntax

```
if (condition)
    code to be executed if condition is true;
elseif (condition)
    code to be executed if condition is true;
else
    code to be executed if condition is false;
```

Example

The following example will output "Have a nice weekend!" if the current day is Friday, and "Have a nice Sunday!" if the current day is Sunday. Otherwise it will output "Have a nice day!":

```
<html>
<body>
<?php
$d=date("D");
if ($d=="Fri")
    echo "Have a nice weekend!";
elseif ($d=="Sun")
    echo "Have a nice Sunday!";
else
    echo "Have a nice day!";
?>
</body>
</html>
```

PHP Switch Statement



The Switch statement in PHP is used to perform one of several different actions based on one of several different conditions.

The Switch Statement

If you want to select one of many blocks of code to be executed, use the Switch statement.

The switch statement is used to avoid long blocks of if..elseif..else code.

Syntax

```
switch (expression)
{
case label1:
    code to be executed if expression = label1;
    break;
case label2:
    code to be executed if expression = label2;
    break;
default:
    code to be executed
    if expression is different
    from both label1 and label2;
}
```

Example

This is how it works:

- A single expression (most often a variable) is evaluated once
- The value of the expression is compared with the values for each case in the structure
- If there is a match, the code associated with that case is executed
- After a code is executed, **break** is used to stop the code from running into the next case
- The default statement is used if none of the cases are true

```
<html>
<body>
<?php
switch ($x)
{
case 1:
    echo "Number 1";
    break;
case 2:
```



```
    echo "Number 2";  
    break;  
case 3:  
    echo "Number 3";  
    break;  
default:  
    echo "No number between 1 and 3";  
}  
?>  
</body>  
</html>
```

PHP Looping

Looping statements in PHP are used to execute the same block of code a specified number of times.

Looping

Very often when you write code, you want the same block of code to run a number of times. You can use looping statements in your code to perform this.

In PHP we have the following looping statements:

- **while** - loops through a block of code if and as long as a specified condition is true
- **do...while** - loops through a block of code once, and then repeats the loop as long as a special condition is true
- **for** - loops through a block of code a specified number of times
- **foreach** - loops through a block of code for each element in an array

The while Statement

The while statement will execute a block of code **if and as long as** a condition is true.

Syntax

```
while (condition)  
code to be executed;
```

Example

The following example demonstrates a loop that will continue to run as long as the variable *i* is less than, or equal to 5. *i* will increase by 1 each time the loop runs:

```
<html>  
<body>  
<?php  
$i=1;  
while ($i<=5)  
{  
    echo "The number is " . $i . "<br />";  
    $i++;  
}  
?>  
</body>  
</html>
```

The do...while Statement

The do...while statement will execute a block of code **at least once** - it then will repeat the loop **as long as** a condition is true.

Syntax

```
do  
{  
  code to be executed;
```

```
}  
while (condition);
```


Example

The following example will increment the value of *i* at least once, and it will continue incrementing the variable *i* as long as it has a value of less than 5:

```
<html>
<body>
<?php
$i=0;
do
{
    $i++;
    echo "The number is " . $i . "<br />";
}
while ($i<5);
?>
</body>
</html>
```

The for Statement

The for statement is the most advanced of the loops in PHP.

In it's simplest form, the for statement is used when you know how many times you want to execute a statement or a list of statements.

Syntax

```
for (init; cond; incr)  
{  
    code to be executed;  
}
```

Parameters:

- **init:** Is mostly used to set a counter, but can be any code to be executed once at the beginning of the loop statement.
- **cond:** Is evaluated at beginning of each loop iteration. If the condition evaluates to TRUE, the loop continues and the code executes. If it evaluates to FALSE, the execution of the loop ends.
- **incr:** Is mostly used to increment a counter, but can be any code to be executed at the end of each loop.



Note: Each of the parameters can be empty or have multiple expressions separated by commas.

- **cond:** All expressions separated by a comma are evaluated but the result is taken from the last part. This parameter being empty means the loop should be run indefinitely. This is useful when using a conditional break statement inside the loop for ending the loop.

Example

The following example prints the text "Hello World!" five times:

```
<html>
<body>
<?php
for ($i=1; $i<=5; $i++)
{
    echo "Hello World!<br />";
}
?>
</body>
</html>
```

The foreach Statement

The foreach statement is used to loop through arrays.

For every loop, the value of the current array element is assigned to \$value (and the array pointer is moved by one) - so on the next loop, you'll be looking at the next element.

Syntax

```
foreach (array as value)  
{  
    code to be executed;  
}
```

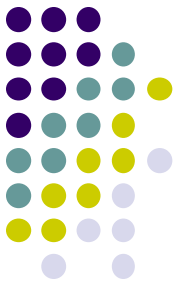
Example

The following example demonstrates a loop that will print the values of the given array:

```
<html>
<body>
<?php
```

```
$arr=array("one", "two", "three");
foreach ($arr as $value)
{
    echo "Value: " . $value . "<br />";
}
?>
</body>
</html>
```

H.W



- Write php code that display hallow message each hour